

Code Appendix

Multivariate Structure and Prediction in the Communities and Crime Dataset

Sidharth Saha Shen-Ching Feng Beckham Wee Joe Wang

STAT 32950 / STAT 24620 / FINM 34700 Spring 2026

All analysis was implemented in Python 3 using `pandas`, `numpy`, `matplotlib`, `seaborn`, and `scikit-learn`. The notebook `communities_crime_report_outline.ipynb` contains the full executed analysis including all cell outputs and figures. The listing below presents the complete analysis code extracted from the notebook, organised by section. AI tools were used for coding assistance and grammar polishing; all analysis design, interpretation, and written explanations are the work of the authors.

```
1  # -----
2  # Setup and Imports
3  # -----
4
5
6  import pandas as pd
7  import numpy as np
8  import matplotlib.pyplot as plt
9  import seaborn as sns
10 import os
11
12 os.environ["OMP_NUM_THREADS"] = "8"
13 os.environ["LOKY_MAX_CPU_COUNT"] = "8"
14 os.environ["MKL_NUM_THREADS"] = "8"
15
16 # -----
17 # Load Data
18 # -----
19
20 crime = pd.read_csv('datasets/communities_crime.csv')
21 df = crime.copy()
22 df.head()
23
24 df.columns.tolist()
25
26
27 # -----
28 # Variable Group Definitions
29 # -----
30
31 variable_groups = {
32     "Population / Urbanization": [
33         "population", "householdsize", "numbUrban", "pctUrban",
34         "LandArea", "PopDens", "PctUsePubTrans"
35     ],
36
37     "Demographic Composition": [
38         "racepctblack", "racePctWhite", "racePctAsian", "racePctHisp",
39         "agePct12t21", "agePct12t29", "agePct16t24", "agePct65up"
40     ],
41
42     "Income / Poverty / Public Assistance": [
43         "medIncome", "medFamInc", "perCapInc",
44         "whitePerCap", "blackPerCap", "indianPerCap", "AsianPerCap",
45         "OtherPerCap", "HispPerCap",
46         "NumUnderPov", "PctPopUnderPov",
47         "pctWWage", "pctWFarmSelf", "pctWInvInc",
48         "pctWSocSec", "pctWPubAsst", "pctWRetire"
49     ],
50
51     "Education / Employment / Occupation": [
52         "PctLess9thGrade", "PctNotHSGrad", "PctBSorMore",
53         "PctUnemployed", "PctEmploy",
54         "PctEmplManu", "PctEmplProfServ",
55         "PctOccupManu", "PctOccupMgmtProf"
56     ]
57 }
```

```

],

"Family Structure": [
    "MalePctDivorce", "MalePctNevMarr", "FemalePctDiv", "TotalPctDiv",
    "PersPerFam", "PctFam2Par", "PctKids2Par",
    "PctYoungKids2Par", "PctTeen2Par",
    "PctWorkMomYoungKids", "PctWorkMom",
    "NumIlleg", "PctIlleg"
],

"Immigration / Language": [
    "NumImmig", "PctImmigRecent", "PctImmigRec5",
    "PctImmigRec8", "PctImmigRec10",
    "PctRecentImmig", "PctRecImmig5",
    "PctRecImmig8", "PctRecImmig10",
    "PctSpeakEnglOnly", "PctNotSpeakEnglWell",
    "PctForeignBorn"
],

"Housing / Residential Stability": [
    "PctLargHouseFam", "PctLargHouseOccup",
    "PersPerOccupHous", "PersPerOwnOccHous", "PersPerRentOccHous",
    "PctPersOwnOccup", "PctPersDenseHous",
    "PctHousLess3BR", "MedNumBR",
    "HousVacant", "PctHousOccup", "PctHousOwnOcc",
    "PctVacantBoarded", "PctVacMore6Mos",
    "MedYrHousBuilt", "PctHousNoPhone", "PctW0FullPlumb",
    "OwnOccLowQuart", "OwnOccMedVal", "OwnOccHiQuart",
    "RentLowQ", "RentMedian", "RentHighQ",
    "MedRent", "MedRentPctHousInc",
    "MedOwnCostPctInc", "MedOwnCostPctIncNoMtg",
    "PctBornSameState", "PctSameHouse85",
    "PctSameCity85", "PctSameState85"
],

"Shelter / Homelessness": [
    "NumInShelters", "NumStreet"
],

"Law Enforcement": [
    "LemasPctOfficDrugUn"
],

"Outcomes": [
    "ViolentCrimesPerPop", "HighViolentCrime"
]
}

variable_group_summary = pd.DataFrame([
    {
        "Variable Group": group,
        "Number of Variables": len(cols),
        "Representative Variables": ", ".join(cols[:5]) + ("..." if len(cols) > 5 else "")
    }
    for group, cols in variable_groups.items()
])

pd.set_option("display.max_colwidth", None)
variable_group_summary

# -----
# Missingness Summary
# -----

missingness_report_table = pd.DataFrame({
    "Missingness Category": [
        "Variables with no missing values",
        "Variables with some missing values",
        "Variables with >20% missingness in retained dataset"
    ],
    "Number of Variables": [

```

```

        (df.isna().sum() == 0).sum(),
        (df.isna().sum() > 0).sum(),
        ((df.isna().mean() * 100) > 20).sum()
    ]
})

missingness_report_table

# -----
# Class Balance
# -----

class_balance = (
    df["HighViolentCrime"]
    .value_counts()
    .rename_axis("Class")
    .reset_index(name="Count")
)

class_balance["Proportion"] = (
    class_balance["Count"] / class_balance["Count"].sum()
).round(3)

class_balance

# -----
# Outcome Distribution
# -----

violent_crime = df["ViolentCrimesPerPop"]

median_crime = violent_crime.median()

plt.figure(figsize=(8, 5))
plt.hist(violent_crime, bins=30, edgecolor="black", alpha=0.8)
plt.axvline(median_crime, linestyle="--", linewidth=2, label=f"Median = {median_crime:.3f}")

plt.title("Figure 2.1: Distribution of ViolentCrimesPerPop")
plt.xlabel("ViolentCrimesPerPop")
plt.ylabel("Number of Communities")
plt.legend()
plt.tight_layout()
plt.show()

violent_crime_skewness = violent_crime.skew()

# -----
# Group-Level Summary Statistics
# -----

group_level_summary = []

for group, cols in variable_groups.items():
    group_data = df[cols]

    group_level_summary.append({
        "Variable Group": group,
        "Number of Variables": len(cols),
        "Average Mean": group_data.mean(numeric_only=True).mean(),
        "Average Std. Dev.": group_data.std(numeric_only=True).mean(),
        "Average Median": group_data.median(numeric_only=True).mean(),
        "Overall Min": group_data.min(numeric_only=True).min(),
        "Overall Max": group_data.max(numeric_only=True).max()
    })

group_level_summary = pd.DataFrame(group_level_summary)

group_level_summary.round(3)

```

```

201 # -----
202 # Crime Outcome Summary
203 # -----
206 violent_crime = df["ViolentCrimesPerPop"]
207
208 crime_summary = pd.DataFrame({
209     "Mean": [violent_crime.mean()],
210     "Std. Dev.": [violent_crime.std()],
211     "Median": [violent_crime.median()],
212     "Min": [violent_crime.min()],
213     "Max": [violent_crime.max()],
214     "Skewness": [violent_crime.skew()]
215 })
216
217 crime_summary.round(3)
218
219 violent_crime = df["ViolentCrimesPerPop"]
220 violent_crime_log = np.log1p(violent_crime)
221
222 fig, axes = plt.subplots(1, 2, figsize=(14, 4))
223
224 # Log-transformed histogram
225 axes[0].hist(violent_crime_log, bins=30, edgecolor="black", alpha=0.8)
226 axes[0].set_title("Log-Transformed Distribution of ViolentCrimesPerPop")
227 axes[0].set_xlabel("log1p(ViolentCrimesPerPop)")
228 axes[0].set_ylabel("Number of Communities")
229
230 # Boxplot of original outcome
231 axes[1].boxplot(violent_crime, vert=False)
232 axes[1].set_title("Boxplot of ViolentCrimesPerPop")
233 axes[1].set_xlabel("ViolentCrimesPerPop")
234
235 plt.tight_layout()
236 plt.show()
237
238 # -----
239 # Correlation Heatmap
240 # -----
241
242 # Representative variables for readable correlation heatmap
243 heatmap_vars = [
244     # Outcome
245     "ViolentCrimesPerPop",
246
247     # Population / urbanization
248     "population", "householdsize", "pctUrban", "PopDens",
249
250     # Demographic composition
251     "racepctblack", "racePctWhite", "racePctHisp", "agePct12t29",
252
253     # Income / poverty
254     "medIncome", "medFamInc", "perCapInc", "PctPopUnderPov", "pctWPubAsst",
255
256     # Education / employment
257     "PctLess9thGrade", "PctNotHSGrad", "PctBSorMore",
258     "PctUnemployed", "PctEmploy",
259
260     # Family structure
261     "MalePctDivorce", "FemalePctDiv", "TotalPctDiv",
262     "PctFam2Par", "PctKids2Par", "PctTeen2Par",
263
264     # Immigration / language
265     "PctForeignBorn", "PctRecentImmig",
266     "PctSpeakEnglOnly", "PctNotSpeakEnglWell",
267
268     # Housing / residential stability
269     "PctHousOwnOcc", "PctPersOwnOccup", "PctHousOccup",
270     "PctVacantBoarded", "PctHousNoPhone",

```

```

    "MedRent", "OwnOccMedVal",
    "PctSameHouse85", "PctSameCity85",

276     # Law enforcement
    "LemasPctOfficDrugUn"
]

# Keep only variables that exist in the dataframe
281 heatmap_vars = [col for col in heatmap_vars if col in df.columns]

corr_matrix = df[heatmap_vars].corr()

plt.figure(figsize=(14, 12))
286 plt.imshow(corr_matrix, aspect="auto")
plt.colorbar(label="Correlation")

plt.xticks(range(len(corr_matrix.columns)), corr_matrix.columns, rotation=90)
plt.yticks(range(len(corr_matrix.index)), corr_matrix.index)

291 plt.title("Figure 3.3: Correlation Heatmap of Selected Variables")
plt.tight_layout()
plt.show()

296 # -----
# EDA Boxplots
# -----

301 # Representative variables for low vs. high violent-crime comparison
boxplot_vars = {
    "medIncome": "Median Household Income",
    "PctPopUnderPov": "Poverty Rate",
    "PctHousOwnOcc": "Owner-Occupied Housing",
306     "PctKids2Par": "Children in Two-Parent Families",
    "PopDens": "Population Density",
    "PctBSorMore": "Bachelor's Degree or More"
}

311 fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()

for ax, (var, label) in zip(axes, boxplot_vars.items()):
    low_values = df[df["HighViolentCrime"] == "Low"][var]
316     high_values = df[df["HighViolentCrime"] == "High"][var]

    ax.boxplot([low_values, high_values], tick_labels=["Low", "High"])
    ax.set_title(label)
    ax.set_xlabel("HighViolentCrime")
321     ax.set_ylabel(var)

plt.suptitle("Figure 3.4: Selected Predictor Distributions by Violent-Crime Class", y=1.02)
plt.tight_layout()
plt.show()

326 # -----
# PCA -- Standardise and Fit
# -----

331 from sklearn.preprocessing import StandardScaler
import pandas as pd

# Define outcome columns
336 outcome_cols = ["ViolentCrimesPerPop", "HighViolentCrime"]

# Predictor matrix only
X = df.drop(columns=outcome_cols)

341 # Standardize predictors for correlation PCA
scaler = StandardScaler()

X_scaled = pd.DataFrame(

```

```

346     scaler.fit_transform(X),
        columns=X.columns,
        index=X.index
    )

    # Quick checks
351 pca_setup_summary = pd.DataFrame({
    "Object": ["Original predictor matrix", "Standardized predictor matrix"],
    "Rows": [X.shape[0], X_scaled.shape[0]],
    "Columns": [X.shape[1], X_scaled.shape[1]],
    "Mean Check": [round(X.mean().mean(), 3), round(X_scaled.mean().mean(), 3)],
356     "Std. Dev. Check": [round(X.std().mean(), 3), round(X_scaled.std().mean(), 3)]
    })

pca_setup_summary

361 # -----
    # PCA -- Scree and Variance
    # -----

366 from sklearn.decomposition import PCA

    pca = PCA()
    pca_scores = pca.fit_transform(X_scaled)

371 pca_scores_df = pd.DataFrame(
    pca_scores,
    columns=[f"PC{i+1}" for i in range(pca_scores.shape[1])],
    index=df.index
    )

376 explained_variance = pca.explained_variance_
    explained_variance_ratio = pca.explained_variance_ratio_
    cumulative_variance = explained_variance_ratio.cumsum()

381 pca_fit_summary = pd.DataFrame({
    "Number of Observations": [X_scaled.shape[0]],
    "Number of Predictors": [X_scaled.shape[1]],
    "Number of PCs Computed": [pca_scores_df.shape[1]],
    "Total Variance Explained": [round(explained_variance_ratio.sum(), 3)]
386 })

pca_fit_summary

391 # -----
    # PCA -- Parallel Analysis and Choosing PCs
    # -----

    # Observed PCA eigenvalues
396 observed_eigenvalues = explained_variance

    # Kaiser rule
    kaiser_pcs = (observed_eigenvalues > 1).sum()

401 # 80% cumulative variance rule
    pcs_for_80 = (cumulative_variance < 0.80).sum() + 1

    # Parallel analysis setup
    n_simulations = 100
406 n_obs, n_vars = X_scaled.shape

    random_eigenvalues = np.zeros((n_simulations, n_vars))

    for i in range(n_simulations):
411         random_data = np.random.normal(size=(n_obs, n_vars))
         random_pca = PCA().fit(random_data)
         random_eigenvalues[i, :] = random_pca.explained_variance_

    random_mean = random_eigenvalues.mean(axis=0)
416 random_95 = np.percentile(random_eigenvalues, 95, axis=0)

```

```

# Parallel analysis rule
parallel_mean_pcs = (observed_eigenvalues > random_mean).sum()
parallel_95_pcs = (observed_eigenvalues > random_95).sum()
421
# Display first 20 PCs in plots
max_pc_plot = 20
pc_numbers = np.arange(1, max_pc_plot + 1)

426 fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Scree plot with Kaiser threshold
axes[0].plot(
    pc_numbers,
    observed_eigenvalues[:max_pc_plot],
    marker="o",
    label="Observed eigenvalues"
)

436 axes[0].axhline(
    y=1,
    linestyle="--",
    linewidth=2,
    label="Kaiser threshold = 1"
441 )

axes[0].set_title("Scree Plot with Kaiser Threshold")
axes[0].set_xlabel("Principal Component")
axes[0].set_ylabel("Eigenvalue")
446 axes[0].set_xticks(pc_numbers)
axes[0].legend()

# Parallel analysis plot
axes[1].plot(
    pc_numbers,
    observed_eigenvalues[:max_pc_plot],
    marker="o",
    label="Observed eigenvalues"
)

456 axes[1].plot(
    pc_numbers,
    random_mean[:max_pc_plot],
    marker="^",
    label="Random mean"
461 )

axes[1].plot(
    pc_numbers,
    random_95[:max_pc_plot],
    linestyle="--",
    label="Random 95%"
466 )

471 axes[1].set_title("Parallel Analysis")
axes[1].set_xlabel("Principal Component")
axes[1].set_ylabel("Eigenvalue")
axes[1].set_xticks(pc_numbers)
axes[1].legend()

476 plt.suptitle("Figure 4.1: Comparing PC-Count Criteria", y=1.03)
plt.tight_layout()
plt.show()

481 pc_count_summary = pd.DataFrame({
    "Criterion": [
        "80% cumulative variance",
        "Kaiser rule: eigenvalue > 1",
        "Parallel analysis: observed > random mean",
        "Parallel analysis: observed > random 95%"
486 ],
    "Number of PCs Suggested": [

```

```

        pcs_for_80,
        kaiser_pcs,
        parallel_mean_pcs,
        parallel_95_pcs
    ]
})

496 pc_count_summary

# -----
# PCA -- Loadings
501 # -----

# PCA loadings for all components
loadings = pd.DataFrame(
    pca.components_.T,
    index=X_scaled.columns,
    columns=[f"PC{i+1}" for i in range(pca.components_.shape[0])]
)

# Side-by-side top loading table for first 4 PCs
511 top_n = 10
pcs_to_interpret = ["PC1", "PC2", "PC3", "PC4"]

side_by_side_loadings = pd.DataFrame()

516 for pc in pcs_to_interpret:
    pc_loadings = loadings[[pc]].copy()
    pc_loadings["Abs Loading"] = pc_loadings[pc].abs()
    pc_loadings = pc_loadings.sort_values("Abs Loading", ascending=False).head(top_n)

521 pc_table = pd.DataFrame({
    f"{pc} Variable": pc_loadings.index,
    f"{pc} Loading": pc_loadings[pc].round(3).values
})

526 side_by_side_loadings = pd.concat(
    [side_by_side_loadings, pc_table.reset_index(drop=True)],
    axis=1
)

531 side_by_side_loadings

# -----
# PCA -- Score Plot
536 # -----

# Combine PCA scores with violent-crime class label
pca_plot_df = pca_scores_df[["PC1", "PC2"]].copy()
pca_plot_df["HighViolentCrime"] = df["HighViolentCrime"]

541 plt.figure(figsize=(10, 5))

for label in ["Low", "High"]:
    subset = pca_plot_df[pca_plot_df["HighViolentCrime"] == label]
546 plt.scatter(
        subset["PC1"],
        subset["PC2"],
        alpha=0.6,
        label=label
551 )

plt.axhline(0, linewidth=1, linestyle="--")
plt.axvline(0, linewidth=1, linestyle="--")

556 plt.title("Figure 4.2: PC1 vs. PC2 Score Plot by Violent-Crime Class")
plt.xlabel("PC1 Score")
plt.ylabel("PC2 Score")
plt.legend(title="HighViolentCrime")
plt.tight_layout()

```



```

561 plt.show()

# -----
# Factor Analysis -- Setup
566 # -----

# Factor analysis uses the same standardized predictor matrix from PCA
X_fa = X_scaled.copy()

571 # Observed predictor correlation matrix (used for diagnostics throughout)
R_observed = X_fa.corr()

# Setup summary
fa_setup_summary = pd.DataFrame({
576     "Object": [
        "Standardized predictor matrix",
        "Observed correlation matrix"
    ],
    "Rows": [X_fa.shape[0], R_observed.shape[0]],
581     "Columns": [X_fa.shape[1], R_observed.shape[1]]
})

fa_setup_summary

586 # -----
# Factor Analysis -- Parallel Analysis and RMSR
# -----

591 from sklearn.decomposition import FactorAnalysis

# -- Parallel analysis --
n_simulations = 100
n_obs, n_vars = X_fa.shape

596 observed_eigenvalues = np.linalg.eigvalsh(R_observed)[:,-1]

random_eigenvalues = np.zeros((n_simulations, n_vars))
for i in range(n_simulations):
601     random_data = np.random.normal(size=(n_obs, n_vars))
    random_corr = pd.DataFrame(random_data).corr()
    random_eigenvalues[i, :] = np.linalg.eigvalsh(random_corr)[:,-1]

random_mean = random_eigenvalues.mean(axis=0)
606 random_95 = np.percentile(random_eigenvalues, 95, axis=0)

parallel_mean_factors = int((observed_eigenvalues > random_mean).sum())
parallel_95_factors = int((observed_eigenvalues > random_95).sum())

611 # -- RMSR across candidate factor counts --
def compute_fa_rmsr(X_data, n_factors):
    """Fit a varimax-rotated FA model and return off-diagonal residual correlation RMSR."""
    fa = FactorAnalysis(n_components=n_factors, rotation="varimax", random_state=42)
    fa.fit(X_data)
616     loadings = fa.components_.T
    R_hat = loadings @ loadings.T + np.diag(fa.noise_variance_)
    d = np.sqrt(np.diag(R_hat))
    R_hat_corr = R_hat / np.outer(d, d)
    residual = R_observed.values - R_hat_corr
621     off_diag = residual[np.triu_indices_from(residual, k=1)]
    return np.sqrt(np.mean(off_diag ** 2))

candidate_range = range(2, 16)
rmsr_results = pd.DataFrame({
626     "Factors": list(candidate_range),
    "RMSR": [compute_fa_rmsr(X_fa, m) for m in candidate_range]
})
rmsr_results["RMSR Improvement"] = rmsr_results["RMSR"].shift(1) - rmsr_results["RMSR"]

631 # -----

```

```

# Factor Analysis -- Diagnostics Plot
# -----

636 # -- Figure 5.1: Factor selection diagnostics (parallel analysis + RMSR) --
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Left: Parallel analysis
max_plot = 20
641 factor_numbers = np.arange(1, max_plot + 1)
axes[0].plot(factor_numbers, observed_eigenvalues[:max_plot], marker="o", label="Observed")
axes[0].plot(factor_numbers, random_mean[:max_plot], marker="~", label="Random mean")
axes[0].plot(factor_numbers, random_95[:max_plot], linestyle="--", label="Random 95th %ile")
axes[0].set_title("Parallel Analysis")
646 axes[0].set_xlabel("Factor Number")
axes[0].set_ylabel("Eigenvalue")
axes[0].set_xticks(factor_numbers)
axes[0].legend()

651 # Right: RMSR curve
axes[1].plot(rmsr_results["Factors"], rmsr_results["RMSR"], marker="o")
axes[1].set_title("Residual Correlation RMSR")
axes[1].set_xlabel("Number of Factors")
axes[1].set_ylabel("RMSR")
656 axes[1].set_xticks(list(candidate_range))

plt.suptitle("Figure 5.1: Factor Selection Diagnostics", y=1.03)
plt.tight_layout()
plt.show()

661 # -----
# Factor Analysis -- Diagnostic Table
# -----

666 # -- Table 5.1: Diagnostic summary --
diagnostic_summary = pd.DataFrame({
    "Criterion": [
        "Parallel analysis (observed > random mean)",
671         "Parallel analysis (observed > random 95th percentile)"
    ],
    "Suggested Factors": [parallel_mean_factors, parallel_95_factors]
})

676 print("Table 5.1a: Parallel Analysis Summary")
display(diagnostic_summary)

print("\nTable 5.1b: RMSR by Factor Count")
display(rmsr_results.round(4))

681 # -----
# Factor Analysis -- Fit Final 7-Factor Model
# -----

686 from sklearn.decomposition import FactorAnalysis

# Fit the 7-factor varimax model directly
final_n_factors = 7
691 final_fa = FactorAnalysis(n_components=final_n_factors, rotation="varimax", random_state=0)
final_fa.fit(X_fa)

# Loading matrix (variables ? factors)
final_loadings = pd.DataFrame(
696     final_fa.components_.T,
    index=X_fa.columns,
    columns=[f"Factor{k+1}" for k in range(final_n_factors)]
)

701 # Factor scores (communities ? factors)
final_scores = pd.DataFrame(
    final_fa.transform(X_fa),
    index=X_fa.index,

```

```

    columns=[f"Factor{k+1}" for k in range(final_n_factors)]
706 )

# Table 5.3: top 8 loadings per factor by absolute value
print(f"Table 5.3: Rotated Factor Loadings ? {final_n_factors}-Factor Model (top 8 per factor)")
for col in final_loadings.columns:
711     top = final_loadings[col].abs().nlargest(8).index
    print(f"\n{col}:")
    print(final_loadings.loc[top, col].round(3).to_string())

716 # -----
# Factor Analysis -- Score Plot
# -----

# Figure 5.2: Factor scores by HighViolentCrime
721 plot_df = final_scores.copy()
plot_df["HighViolentCrime"] = df["HighViolentCrime"].values

n_factors = final_scores.shape[1]
fig, axes = plt.subplots(1, n_factors, figsize=(3 * n_factors, 5), sharey=False)
726

for i, factor_col in enumerate(final_scores.columns):
    low_values = plot_df[plot_df["HighViolentCrime"] == "Low"][factor_col]
    high_values = plot_df[plot_df["HighViolentCrime"] == "High"][factor_col]

731     axes[i].boxplot(
        [low_values, high_values],
        tick_labels=["Low", "High"]
    )

736     axes[i].set_title(factor_col)
    axes[i].set_xlabel("HighViolentCrime")
    axes[i].set_ylabel("Factor Score")

plt.suptitle("Figure 5.2: Factor Scores by Violent-Crime Class", y=1.03)
741 plt.tight_layout()
plt.show()

# -----
746 # Factor Analysis -- Score Comparison
# -----

factor_score_comparison = []

751 for factor in final_scores.columns:
    low_scores = plot_df[plot_df["HighViolentCrime"] == "Low"][factor]
    high_scores = plot_df[plot_df["HighViolentCrime"] == "High"][factor]

    factor_score_comparison.append({
756         "Factor": factor,
        "Low Mean": low_scores.mean(),
        "High Mean": high_scores.mean(),
        "Mean Difference: High - Low": high_scores.mean() - low_scores.mean(),
        "Low Median": low_scores.median(),
761         "High Median": high_scores.median(),
        "Median Difference: High - Low": high_scores.median() - low_scores.median()
    })

factor_score_comparison = pd.DataFrame(factor_score_comparison)
766

factor_score_comparison["Abs Mean Difference"] = (
    factor_score_comparison["Mean Difference: High - Low"].abs()
)

771 factor_score_comparison_sorted = factor_score_comparison.sort_values(
    "Abs Mean Difference",
    ascending=False
)

776 factor_score_comparison_sorted.round(3)

```

```

# Clustering input setup: retained PCA scores

# Number of PCs needed to reach at least 80% cumulative variance
781 n_pcs_clustering = pcs_for_80

# Retained PCA score matrix for clustering
cluster_input = pca_scores_df.iloc[:, :n_pcs_clustering].copy()

786 # Summary table
cluster_input_summary = pd.DataFrame({
    "Input Representation": ["Retained PCA scores"],
    "Distance Geometry": ["Euclidean"],
    "Number of Observations": [cluster_input.shape[0]],
791 "Number of Features": [cluster_input.shape[1]],
    "Selection Rule": ["Minimum PCs needed to reach 80% cumulative variance"],
    "Cumulative Variance Explained": [cumulative_variance[n_pcs_clustering - 1]]
})

796 cluster_input_summary.round(3)

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import pandas as pd
801 import os

os.environ["OMP_NUM_THREADS"] = "8"
os.environ["LOKY_MAX_CPU_COUNT"] = "8"

806 # Candidate K values
k_values = range(2, 21)

kmeans_results = []

811 for k in k_values:
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=20
816    )

    cluster_labels = kmeans.fit_predict(cluster_input)

    kmeans_results.append({
821         "K": k,
        "Within-Cluster Sum of Squares": kmeans.inertia_,
        "Silhouette Score": silhouette_score(cluster_input, cluster_labels)
    })

826 kmeans_results_table = pd.DataFrame(kmeans_results)

kmeans_results_table.round(3)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

831 # Figure 6.1: Elbow plot
axes[0].plot(
    kmeans_results_table["K"],
    kmeans_results_table["Within-Cluster Sum of Squares"],
836    marker="o"
)

axes[0].set_title("Figure 6.1: K-Means Elbow Plot")
axes[0].set_xlabel("Number of Clusters K")
841 axes[0].set_ylabel("Within-Cluster Sum of Squares")
axes[0].set_xticks(kmeans_results_table["K"])

# Figure 6.2: Silhouette plot
846 axes[1].plot(
    kmeans_results_table["K"],
    kmeans_results_table["Silhouette Score"],
    marker="o"

```

```

)

851 axes[1].set_title("Figure 6.2: Average Silhouette Score")
    axes[1].set_xlabel("Number of Clusters K")
    axes[1].set_ylabel("Silhouette Score")
    axes[1].set_xticks(kmeans_results_table["K"])

856 plt.tight_layout()
    plt.show()

    # Final K-means model
    selected_k = 3

861 final_kmeans = KMeans(
    n_clusters=selected_k,
    random_state=42,
    n_init=20

866 )

    kmeans_labels = final_kmeans.fit_predict(cluster_input)

    # Create clustering results dataframe
871 kmeans_cluster_df = pca_scores_df.copy()
    kmeans_cluster_df["KMeans Cluster"] = kmeans_labels
    kmeans_cluster_df["ViolentCrimesPerPop"] = df["ViolentCrimesPerPop"].values
    kmeans_cluster_df["HighViolentCrime"] = df["HighViolentCrime"].values

876 # Convert High/Low label to numeric indicator for cluster summaries
    kmeans_cluster_df["HighViolentCrime Indicator"] = (
        kmeans_cluster_df["HighViolentCrime"] == "High"
    ).astype(int)

881 # Cluster profile table using first 4 PCs for interpretation
    pc_profile_cols = ["PC1", "PC2", "PC3", "PC4"]

    kmeans_profile_table = (
        kmeans_cluster_df
886 .groupby("KMeans Cluster")
        .agg(
            Number_of_Communities=("KMeans Cluster", "size"),
            Mean_ViolentCrimesPerPop=("ViolentCrimesPerPop", "mean"),
            Proportion_HighViolentCrime=("HighViolentCrime Indicator", "mean"),
891 Mean_PC1=("PC1", "mean"),
            Mean_PC2=("PC2", "mean"),
            Mean_PC3=("PC3", "mean"),
            Mean_PC4=("PC4", "mean")
        )
896 .reset_index()
    )

    # Sort clusters from lowest to highest average violent crime for easier interpretation
    kmeans_profile_table = kmeans_profile_table.sort_values(
901 "Mean_ViolentCrimesPerPop"
    ).reset_index(drop=True)

    kmeans_profile_table.round(3)

906 # PC-space visualization dataframe
    kmeans_plot_df = kmeans_cluster_df[["PC1", "PC2", "KMeans Cluster"]].copy()

    plt.figure(figsize=(9, 6))

911 for cluster in sorted(kmeans_plot_df["KMeans Cluster"].unique()):
    subset = kmeans_plot_df[kmeans_plot_df["KMeans Cluster"] == cluster]
    plt.scatter(
        subset["PC1"],
        subset["PC2"],
916 alpha=0.6,
        label=f"Cluster {cluster}"
    )

    plt.axhline(0, linestyle="--", linewidth=1)

```

```

921 plt.axvline(0, linestyle="--", linewidth=1)

plt.title("Figure 6.3: K-Means Clusters in PC1?PC2 Space")
plt.xlabel("PC1 Score")
plt.ylabel("PC2 Score")
926 plt.legend(title="K-Means Cluster")
plt.tight_layout()
plt.show()

931 # -----
# Classification -- Model Fitting and Evaluation
# -----

from sklearn.model_selection import train_test_split
936 from sklearn.preprocessing import StandardScaler

# Binary outcome
y = (df["HighViolentCrime"] == "High").astype(int)

941 # Input 1: original standardized predictors
X_original = X_scaled.copy()

# Input 2: retained PCA scores
X_pca_classification = pca_scores_df.iloc[:, :pcs_for_80].copy()
946 # Input 3: factor scores from final factor model
X_factor_classification = final_scores.copy()

# Train/test split using same indices for all inputs
951 train_idx, test_idx = train_test_split(
    df.index,
    test_size=0.30,
    random_state=42,
    stratify=y
956 )

X_original_train = X_original.loc[train_idx]
X_original_test = X_original.loc[test_idx]

961 X_pca_train = X_pca_classification.loc[train_idx]
X_pca_test = X_pca_classification.loc[test_idx]

X_factor_train = X_factor_classification.loc[train_idx]
X_factor_test = X_factor_classification.loc[test_idx]
966 y_train = y.loc[train_idx]
y_test = y.loc[test_idx]

classification_setup = pd.DataFrame({
971     "Input": [
        "Original standardized predictors",
        "Retained PCA scores",
        "Factor scores"
    ],
976     "Train Rows": [
        X_original_train.shape[0],
        X_pca_train.shape[0],
        X_factor_train.shape[0]
    ],
981     "Test Rows": [
        X_original_test.shape[0],
        X_pca_test.shape[0],
        X_factor_test.shape[0]
    ],
986     "Number of Features": [
        X_original_train.shape[1],
        X_pca_train.shape[1],
        X_factor_train.shape[1]
    ]
991 })

```

```

classification_setup

996 # -----
# Classification -- ROC Curves
# -----

from sklearn.linear_model import LogisticRegression
1001 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    roc_auc_score,
1006 roc_curve
)

def evaluate_classifier(model_name, model, X_train, X_test, y_train, y_test):
    model.fit(X_train, y_train)

1011 y_pred = model.predict(X_test)

    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(X_test)[:, 1]
1016 else:
        y_prob = model.decision_function(X_test)

    tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()

1021 accuracy = accuracy_score(y_test, y_pred)
    sensitivity = tp / (tp + fn)
    specificity = tn / (tn + fp)
    auc = roc_auc_score(y_test, y_prob)

1026 return {
    "Model": model_name,
    "Accuracy": accuracy,
    "Sensitivity": sensitivity,
    "Specificity": specificity,
1031 "ROC-AUC": auc,
    "TN": tn,
    "FP": fp,
    "FN": fn,
    "TP": tp,
1036 "Fitted Model": model,
    "Predicted Probabilities": y_prob
}

models_to_evaluate = [
1041 (
    "Logistic Regression: Original Predictors",
    LogisticRegression(max_iter=5000),
    X_original_train,
    X_original_test
1046 ),
    (
    "Logistic Regression: PCA Scores",
    LogisticRegression(max_iter=5000),
    X_pca_train,
    X_pca_test
1051 ),
    (
    "Logistic Regression: Factor Scores",
    LogisticRegression(max_iter=5000),
1056 X_factor_train,
    X_factor_test
    ),
    (
    "LDA: PCA Scores",
1061 LinearDiscriminantAnalysis(),
    X_pca_train,
    X_pca_test
    ),
]

```

```

1066     "LDA: Factor Scores",
        LinearDiscriminantAnalysis(),
        X_factor_train,
        X_factor_test
    )
1071 ]

classification_results = []

for model_name, model, X_train_model, X_test_model in models_to_evaluate:
1076     result = evaluate_classifier(
        model_name,
        model,
        X_train_model,
        X_test_model,
1081         y_train,
        y_test
    )
    classification_results.append(result)

1086 classification_results_table = pd.DataFrame(classification_results).drop(
    columns=["Fitted Model", "Predicted Probabilities"]
)

classification_results_table[
1091     ["Model", "Accuracy", "Sensitivity", "Specificity", "ROC-AUC"]
].round(3)

# -----
1096 # Classification -- Best Model Confusion Matrix
# -----

plt.figure(figsize=(8, 6))

1101 for result in classification_results:
    fpr, tpr, thresholds = roc_curve(
        y_test,
        result["Predicted Probabilities"]
    )

1106     plt.plot(
        fpr,
        tpr,
        label=f'{result["Model"]} (AUC = {result["ROC-AUC"]:.3f})'
1111     )

plt.plot([0, 1], [0, 1], linestyle="--")

plt.title("Figure 7.1: ROC Curves for Classification Models")
1116 plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.tight_layout()
plt.show()

1121 # Select best model by ROC-AUC
best_result = max(classification_results, key=lambda x: x["ROC-AUC"])

best_confusion_matrix = pd.DataFrame(
1126     [[best_result["TN"], best_result["FP"]],
     [best_result["FN"], best_result["TP"]]],
     index=["Actual Low", "Actual High"],
     columns=["Predicted Low", "Predicted High"]
)

1131 print("Best model:", best_result["Model"])
best_confusion_matrix

1136 # -----

```



```

# Regularisation -- Setup
# -----

from sklearn.linear_model import LogisticRegressionCV
1141 from sklearn.metrics import accuracy_score, confusion_matrix, roc_auc_score
import pandas as pd
import numpy as np

# Regularized logistic regression uses the original standardized predictors
1146 X_reg_train = X_original_train.copy()
X_reg_test = X_original_test.copy()

# C is inverse regularization strength:
# smaller C = stronger regularization
1151 C_values = np.logspace(-4, 4, 50)

regularization_setup = pd.DataFrame({
    "Input": ["Original standardized predictors"],
    "Train Rows": [X_reg_train.shape[0]],
1156 "Test Rows": [X_reg_test.shape[0]],
    "Number of Features": [X_reg_train.shape[1]],
    "C Values Tested": [len(C_values)],
    "Cross-Validation Folds": [5]
})
1161 regularization_setup

# -----
1166 # Regularisation -- Ridge, Lasso, Elastic Net
# -----

# Ridge logistic regression: L2 penalty
ridge_cv = LogisticRegressionCV(
1171     Cs=C_values,
    cv=5,
    penalty="l2",
    solver="lbfgs",
    scoring="roc_auc",
1176     max_iter=5000,
    random_state=42
)

# Lasso logistic regression: L1 penalty
1181 lasso_cv = LogisticRegressionCV(
    Cs=C_values,
    cv=5,
    penalty="l1",
    solver="saga",
1186     scoring="roc_auc",
    max_iter=5000,
    random_state=42,
    n_jobs=-1
)
1191

# Elastic net logistic regression: combined L1/L2 penalty
elastic_net_cv = LogisticRegressionCV(
    Cs=C_values,
    cv=5,
1196     penalty="elasticnet",
    solver="saga",
    l1_ratios=[0.25, 0.50, 0.75],
    scoring="roc_auc",
    max_iter=5000,
1201     random_state=42,
    n_jobs=-1
)

regularized_models = {
1206     "Ridge Logistic Regression": ridge_cv,
    "Lasso Logistic Regression": lasso_cv,
    "Elastic Net Logistic Regression": elastic_net_cv
}

```

```

}

1211 for model in regularized_models.values():
    model.fit(X_reg_train, y_train)

def evaluate_regularized_model(model_name, model, X_test, y_test):
    y_pred = model.predict(X_test)
1216 y_prob = model.predict_proba(X_test)[: , 1]

    tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()

    coef = model.coef_.ravel()
1221 nonzero_count = (coef != 0).sum()

    return {
        "Model": model_name,
        "Accuracy": accuracy_score(y_test, y_pred),
1226 "Sensitivity": tp / (tp + fn),
        "Specificity": tn / (tn + fp),
        "ROC-AUC": roc_auc_score(y_test, y_prob),
        "Selected C": model.C_[0],
        "Nonzero Coefficients": nonzero_count
1231     }

regularized_results = []

for model_name, model in regularized_models.items():
1236     regularized_results.append(
        evaluate_regularized_model(
            model_name,
            model,
            X_reg_test,
1241 y_test
        )
    )

regularized_results_table = pd.DataFrame(regularized_results)
1246 regularized_results_table.round(3)

def coefficient_table(model, feature_names, model_name):
    coef = model.coef_.ravel()

1251     coef_df = pd.DataFrame({
        "Variable": feature_names,
        "Coefficient": coef,
        "Abs Coefficient": np.abs(coef),
1256 "Model": model_name
    })

    coef_df = coef_df[coef_df["Coefficient"] != 0]
    coef_df = coef_df.sort_values("Abs Coefficient", ascending=False)
1261     return coef_df

lasso_coef_table = coefficient_table(
    lasso_cv,
1266 X_reg_train.columns,
    "Lasso Logistic Regression"
)

elastic_net_coef_table = coefficient_table(
    elastic_net_cv,
1271 X_reg_train.columns,
    "Elastic Net Logistic Regression"
)

1276 lasso_coef_table.round(3).head(20)

# Mean cross-validated ROC-AUC for lasso across C values
lasso_mean_auc = lasso_cv.scores_[1].mean(axis=0)

```

```

1281 plt.figure(figsize=(8, 5))

    plt.plot(
        lasso_cv.Cs_,
        lasso_mean_auc,
1286     marker="o"
    )

    plt.xscale("log")
    plt.axvline(
1291     lasso_cv.C_[0],
        linestyle="--",
        label=f"Selected C = {lasso_cv.C_[0]:.4f}"
    )

1296 plt.title("Figure 8.1: Lasso Cross-Validated ROC-AUC")
    plt.xlabel("C (Inverse Regularization Strength)")
    plt.ylabel("Mean Cross-Validated ROC-AUC")
    plt.legend()
    plt.tight_layout()
1301 plt.show()

```